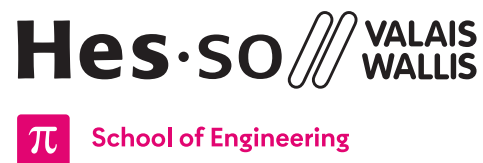




Game of Life (GoL)

Lecture Computer Architecture (CAr)



Orientation: [Information and Communication Technology \(ISC\)](#)

Specialisation: Data Engineering (DE)

Course: [Computer Architecture \(CAr\)](#)

Authors: [Silvan Zahno](#), [Axel Amand](#), [Rémy Borgeat](#)

Date: 05.08.2026

Version: v1.0



Contents

1 Introduction	3
2 Specification	4
2.1 Minimal specification	6
2.2 Rules	6
2.2.1 Toroidal grid	8
2.2.2 Tips	8
3 Software	9
3.1 Grids and cells representation	9
3.1.1 Grid	9
3.1.2 Cell	9
3.2 Tasks	10
3.2.1 gol_init	10
3.2.2 gol_step	10
3.3 Compile and flash	11
3.4 Testing	12
3.4.1 Hardware In The Loop (HITL)	12
3.5 GoL Viewer usage	13
3.5.1 Setup	13
3.5.2 Viewing the grid	14
3.5.3 Editing the grid	14
4 Evaluation	15
Appendix	16
1 Debugging (Optional)	16
1.1 Setup	16
1.1.1 Extensions	16
1.1.2 Drivers	16
1.1.3 Debug	16
Glossary	20



1 Introduction

The aim of the project is to apply the know [Light Emitting Diode \(LED\)](#) ge acquired at the end of the semester directly using a practical example.

The goal is to implement [Conway's Game of Life \(GoL\)](#) , a cellular automaton system based on 4 simple rules, acting "by itself" - *also known as a zero-player game* -.

The [Game of Life](#) has been created by the British mathematician John Conway in 1970.

A grid is composed of cells, each of which can be either alive or dead. The evolution of the grid is based on 4 simple rules applied to each cell based on its neighbors - see [Section 2.2](#).

The game uses a desktop application to create a grid of cells then sent to an [ESP-RS devboard](#).

The microcontroller applies the rules of the game to the grid and sends back the result to the application to view the system evolving live. The red [LED](#) on the board toggles each time the grid is updated to show it is running.

This system is represented in the illustration [Figure 1](#):

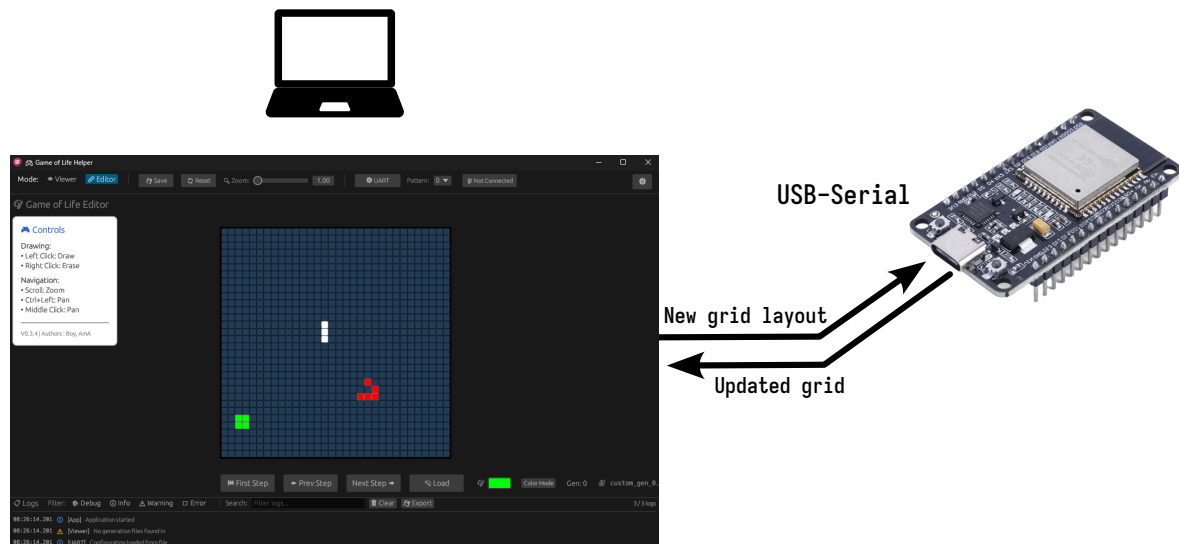


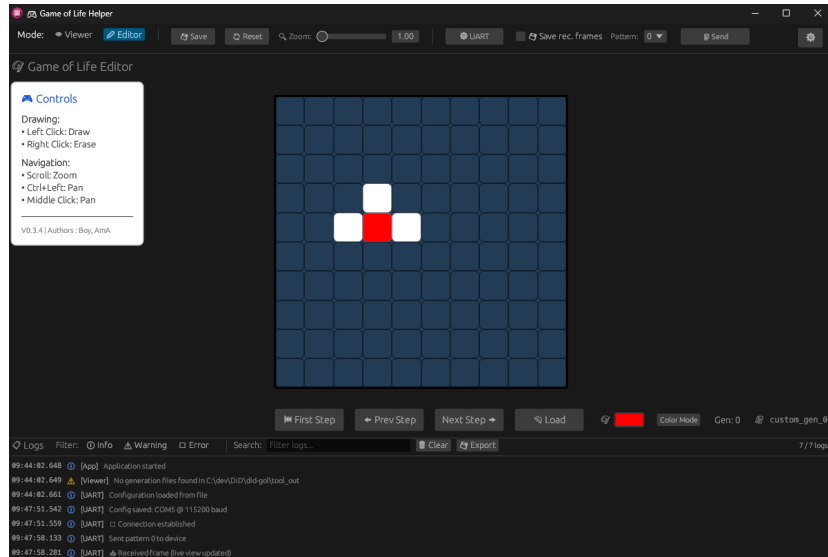
Figure 1 - GoL setup

The task is to implement the grid generation using assembly language according to the specification [Section 2](#).



2 Specification

The system uses the **GoL Viewer**, a desktop application, to create and view grids of cells



The cells are then sent to an **ESP-RS devboard**.

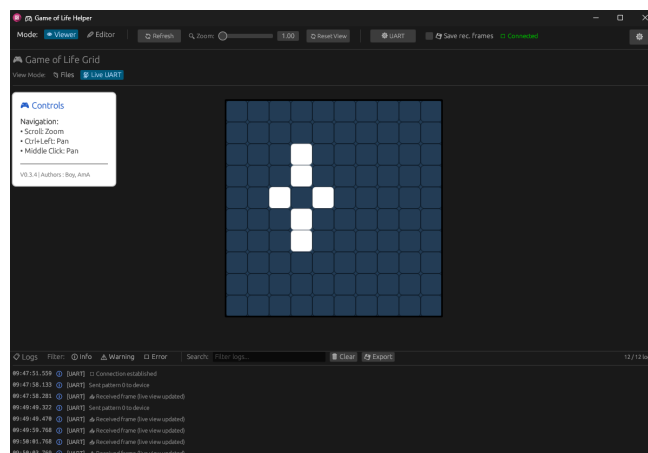
This board embeds an **ESP32-C3** chip hosting a single core, 32 bits **5-stages Reduced Instruction Set Computer architecture, open-sourced (RISC-V)** microcontroller supporting the **RV32IMC** instruction sets.

It runs an **Embedded Rust** based application, using Rust and partially **RISC-V** assembly code at keypoints as shown in the diagram **Figure 3**.

The assembly code is split in **two parts**:

1. **gol_init**: setup a default grid by the ESP32-C3 microcontroller
2. **gol_step**: receives a grid from the **GoL Viewer** and must apply **GOL** rules to it, then send it to the viewer

When grids are received, the **GoL Viewer** application displays it on the screen:



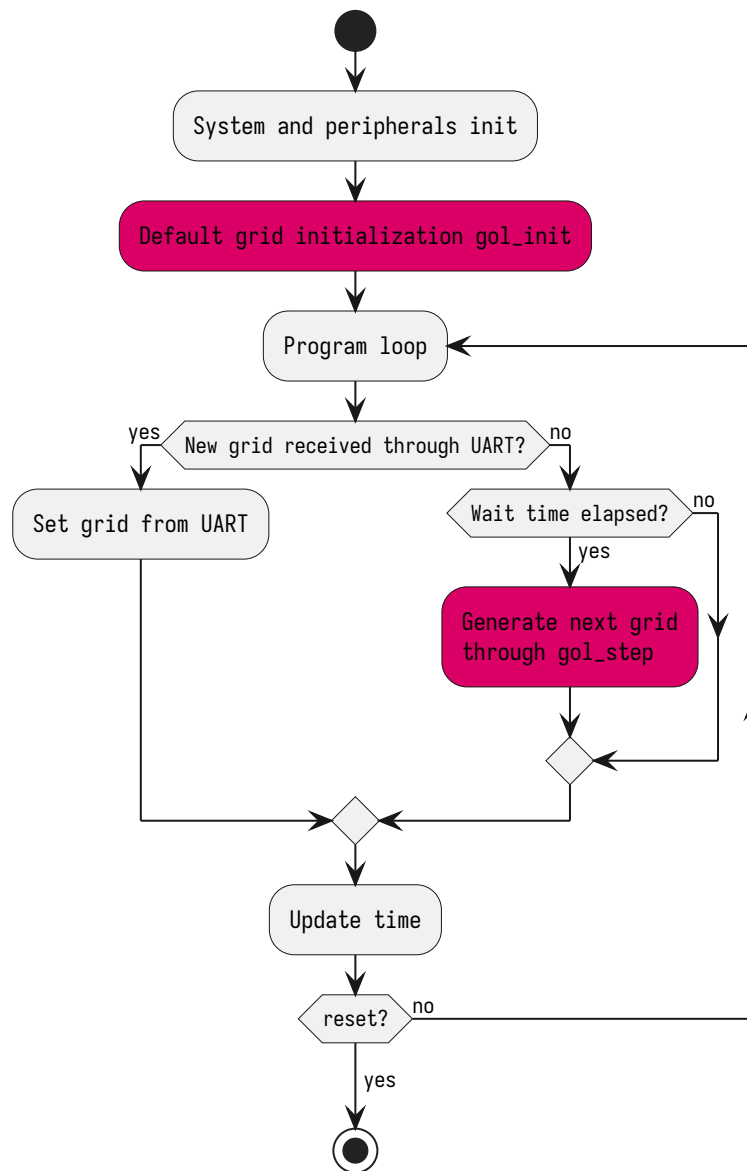


Figure 3 - System flow diagram for the Game of Life implementation



The **C** extension of the **RV32IMC** defines compressed instructions, particularly useful on embedded systems with limited memory. In this project, this **Instruction Set Architecture (ISA)** can be ignored while writing direct assembly code.



2.1 Minimal specification

The system must implement the following functionalities:

- At startup in `gol_init`, the system must display a default grid pattern on the screen without any user interaction from the **GoL Viewer**. The grid must be 32x32 cells and contain a *blinker pattern* in the center of the gamefield - [Figure 4](#).
- At each `gol_step` call, the grid must be updated according to the **GoL** rules [Section 2.2](#) and sent back to the **GoL Viewer** application for display. The rules must be applied in a *toroidal* manner, meaning that the grid wraps around its edges - see [Section 2.2.1](#).
- The code must follow conventions and make use of function calls to avoid code duplication and improve readability and debugging. The code must be commented to explain functions and logic behind the implementation.
- A report must be written according to [Section 4](#).
- A graphical summary of the code flow must be provided similar to [Figure 2](#), showing the main steps of the `gol_init` with decisions and function calls.
- A graphical summary of the code flow must be provided similar to [Figure 2](#), showing the main steps of the `gol_step` with decisions and function calls.
- There is no AI generated content allowed in the code or report. The code and report must be written by the student.

2.2 Rules

GoL is a cellular automaton system which consists of a grid of cells, where each one is either *alive* or *dead*. The evolution of the grid follows four simple rules based on the current cell and its surrounding neighbors:

1. Underpopulation

Any live cell with fewer than two (< 2) live neighbors *dies*



2. Survival

Any live cell with two ($= 2$) live neighbors *survives* to the next generation



3. Overpopulation

Any live cell with more than three (> 3) live neighbors *dies*



4. Reproduction

Any dead cell with exactly three ($= 3$) live neighbors becomes *alive*



These simple rules create fascinating emergent behaviors including stable patterns, oscillators, and moving structures called “gliders” as shown in [Figure 4](#):

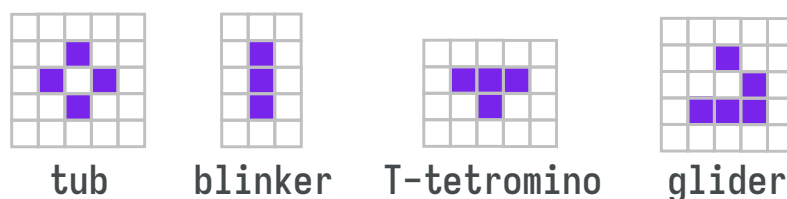


Figure 4 - Interesting patterns in **GoL** : stable structures, oscillators, and gliders



The game can be experimented under <https://playgameoflife.com> or <https://lifeee.netlify.app>.



2.2.1 Toroidal grid

A **toroidal** grid is a grid where the edges wrap around, creating an “infinite” surface without boundaries.

Applied to this project, this means some neighbors of cells on the extremities of the grid appear on the opposite side

As example, let's take the following 6x6 grid and check neighbors of cell (0,0):

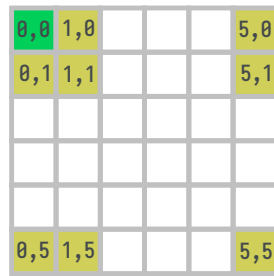


Figure 5 - Neighbours of cell (0, 0)

1. Left neighbors overflow to the right of the grid
2. Top neighbors overflow to the bottom of the grid
3. Top-left neighbor overflows to the bottom-right of the grid

Generalizing this, the neighbors of a cell at position (x, y) in a grid of size (X_Size, Y_Size) are:

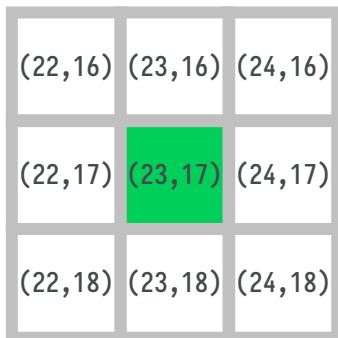


Figure 6 - Neighbours of cell (23, 17)

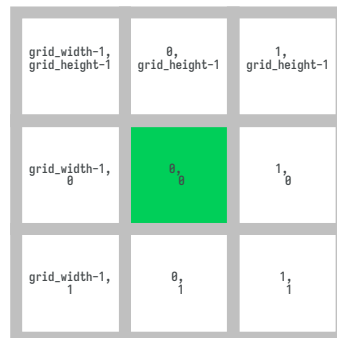


Figure 7 - Neighbours of cell (0, 0)

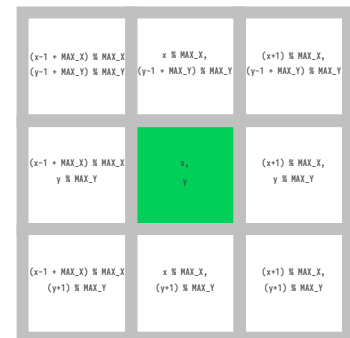


Figure 8 - Neighbours of a generic cell (x, y)

2.2.2 Tips

1. Begin by testing the [GoL](#) games on the web to understand the rules and how they work.
2. Start by defining code flow, functions, decisions ... and draw a diagram similar to [Figure 2](#). This will help you understand the system and how to implement it.
3. Implement the `gol_init` function to play with reads and writes in memory. Replace the `unsafe { gol_step(cur, nxt, COLS as u32, ROWS as u32) };` line in `main.rs` with `rust_send_grid(cur as *const u32, CELLS as u32);` to display and view this grid in the GoL Viewer.
4. Restore the rust `unsafe { gol_step...` code and implement the `gol_step` function to apply the rules of the game to the grid. Go in small steps and test often by running your code and viewing evolution in the GoL Viewer. If needed, iterate bit by bit with the debugger - see [Section 1](#).
5. Once everything is in place, run the tests to ensure both generation and toroidal rules are correctly implemented - see [Section 3.4](#).



3 Software



The [Assembly \(ASM\)](#) code to implement is under `src/asm/gol.s` and is called by the Rust code in `src/bin/main.rs`.

3.1 Grids and cells representation

Since the assembly code is mixed with pre-given Rust code, it is important to understand some important notions first.

3.1.1 Grid

The grids are defined in `grid.rs` as:

```
pub static mut GRID_X: [u32; WIDTH * HEIGHT] = [0u32; WIDTH * HEIGHT];
```

In memory, a grid is stored as a contiguous 1D array of **Width * Height** cells, but can be seen as a 2D array where each cell is given by its (**x**, **y**) coordinates.

Let's take an example of a 3x3 cells grid:

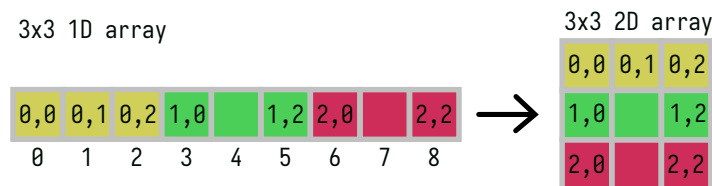


Figure 9 - Grid in memory



Internally, two grids **A** and **B** are used, notably for [Section 3.2.2](#). This allows to compute the next generation of the grid in **B** while reading the current generation in **A** without losing data. Once the computation is done, the two grids are swapped.

3.1.2 Cell

As seen in [previous snippet 1](#), the grid is defined as an array of 32-bit unsigned integers.

Each cell is actually represented by a 32bit value `xxRRGGBB`, where `xx` is a reserved byte (fixed to `0x00`), and `RRGGBB` is the color of the cell in hexadecimal format.

This color is also handled by the **GoL Viewer** and can be used for advanced functionalities like implementing a [Rainbow Game of Life](#) where each cell inherits colors when being born.

A **dead** cell is determined by having all its **Red Green Blue (RGB)** bits set to 0, while all other **RGB** values are considered as **alive** cells:

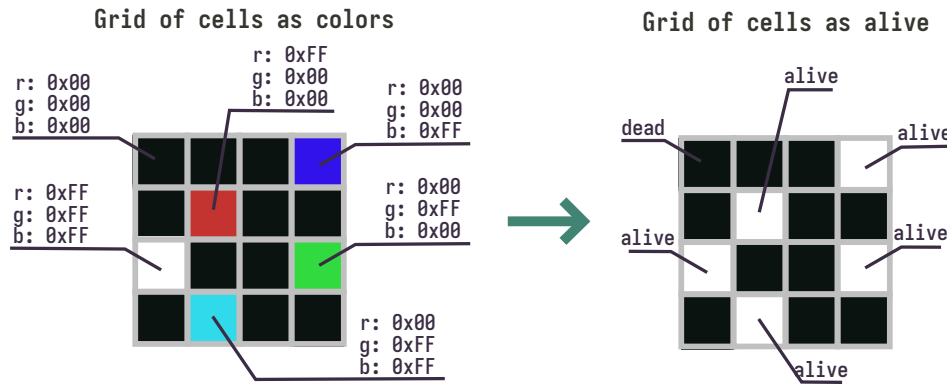


Figure 10 - Cell alive status

3.2 Tasks

3.2.1 gol_init

The **gol_init** function is called at startup and must initialize the grid with the default pattern - a *blinker* in the center of the grid - [Figure 4](#).

It is defined as **fn gol_init(data_ptr: *mut u32, cols: u32, rows: u32)**; where:

- **data_ptr** contains the memory address of the grid, cell (0, 0). Be aware to access only allowed memory locations since there is no boundary check.
- **cols** is the total number of columns in the grid
- **rows** is the total number of rows in the grid



1. Draw a flow diagram presenting the steps to initialize the grid with a blinker pattern in the center.
2. Write asm code to iterate the whole grid to decide whether to set a cell as **dead** (0xXX000000) or **alive** (any other value depending if colors are used or not).
3. Initialize the grid pattern at startup by placing a blinker - [Figure 4](#) - in its center.

3.2.2 gol_step

The **gol_step** function is called each time the next generation of the grid must be computed.

It is defined as **fn gol_step(in_data_ptr: *mut u32, out_data_ptr: *mut u32, cols: u32, rows: u32)**; where:

- **in_data_ptr** contains the memory address of the input grid, cell (0, 0). Be aware to access only allowed memory locations since there is no boundary check. Only read from it.
- **out_data_ptr** contains the memory address of the output grid, cell (0, 0). Be aware to access only allowed memory locations since there is no boundary check. Only write to it.
- **cols** is the total number of columns in the grid
- **rows** is the total number of rows in the grid

The recipe to compute the next generation is as follows:

1. Iterate over all input cells. For each:
 - Iterate over all neighbors of the current cell, counting how many are **alive**



- Apply the rules of [Section 2.2](#) to determine whether the current cell is alive or dead in the next generation
 - Register the new value in the output grid at the same coordinates as the current cell
2. Once the grid is fully computed, send it through [Universal Asynchronous Receiver Transmitter \(UART\)](#) by calling the helper function `fn rust_send_grid(ptr: *const u32, count: u32);` where:
- `ptr` contains the memory address of the output grid, cell `(0, 0)`
 - `count` is the total number of cells in the grid, i.e. `Width * Height`



To call this function, respect the [RISCV](#) calling convention. You can then use a call to label `rust_send_grid` to access the function.

3. Finally, swap the **A** and **B** grids to prepare for the next generation computation by calling the helper function `fn rust_swap_grids();`.



To call this function, respect the [RISCV](#) calling convention. You can then use a call to label `rust_swap_grids` to access the function.



1. Draw a flow diagram presenting the steps to compute the next generation of the grid.
2. Compute the next generation of the grid and send it through [UART](#).

3.3 Compile and flash

To release the GoL software, use the given **justfile** to compile and flash the [RISCV](#) core:

```
# One by one
just gol-flash # Compile and flash the GoL software to the RISCV core
just gol-viewer # Launch the GoL Viewer to visualize the grid

# Both together
just run # Compile and flash the GoL software to the RISCV core,
        # then launch the GoL Viewer
```



3.4 Testing

Due to the limited debugging capabilities - see [Section 1](#), only full-software tests can be automated - see [Section 3.4.1](#). The rest of the code can be implemented with the tools seen during the laboratories or by flashing the [Central Processing Unit \(CPU\)](#) and checking the results in the viewer.

3.4.1 HITL

Final tests are performed in a [HITL](#) environment, where the [RISCV](#) core is running the GoL software while a dedicated PC application is used to set and get results through [UART](#) and evaluate the correctness of the software.

To run those tests, use the given **justfile** to launch the [HITL](#) environment:

```
just test <port> # Compile and flash the #gls("riscv") core then launch the HITL
application to test the GoL software
# Port on Windows should be COMx, on Unix /dev/ttyUSBx or /dev/ttyACMx.
# You can also set the DEF_PORT variable in the justfile to avoid specifying the port
each time.
```

This test checks:

1. When freshly flashed, the grid should already contain a default pattern containing a blinker in the center of the grid
2. Then, it sends known grids and check a few generations:
 - Block (still pattern)
 - Beehive (still pattern)
 - Blinker (period-2 oscillator)
 - Pulsar (period-3 oscillator)
 - Glider (moving pattern)

After the tests, log files are generated near the **justfile** to provide detailed reports of the tests and their results along a summary of tests:

```
Total tests: 6
Passed: 6
Failed: 0
grid_init: PASS
block: PASS
beehive: PASS
blinker: PASS
pulsar: PASS
glider: PASS
```

```
HIL summary started: port=COM3,
baud=115200 (summary log:
<path>\<to>\test_summary.log)
grid_init log:
<path>\<to>\test_grid_init.log
grid_init PASS
pattern=block PASS
pattern=beehive PASS
pattern=blinker PASS
pattern=pulsar PASS
pattern=glider PASS
HIL result: PASS (5 passed, 0 failed)
```



3.5 GoL Viewer usage

The GoL Viewer is a simple application to set and visualize the grid and its evolution. It is written in **rust** and links to the **CPU** through **UART**. It is available for Windows, Linux and MacOS.

The viewer is automatically launched from the recipes **just run** or **just gol-viewer**. It can also be launched manually by running the **gol-viewer** executable in the **GoLViewer** folder of the project:

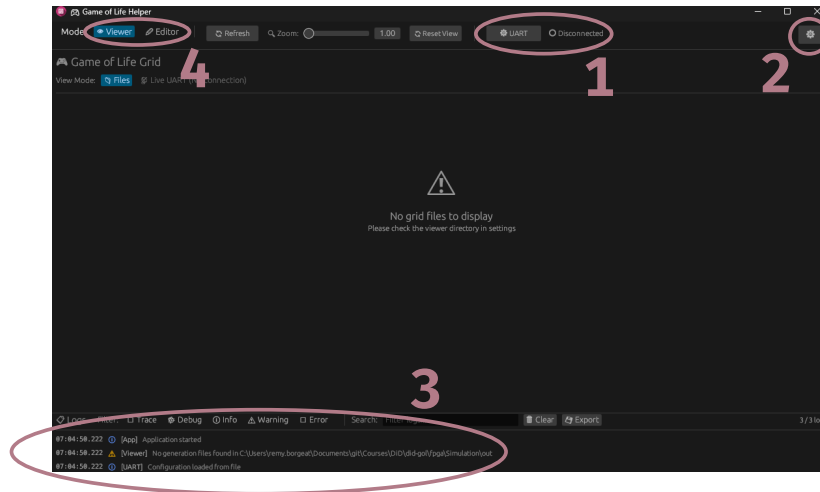


Figure 11 - GoL Viewer

1. **UART** connection status and configuration
2. viewer and grid settings
3. log window
4. view / edit switch

3.5.1 Setup

Start by plugging the ESP board and then setting the correct **UART** port. Without special configuration, the biggest port from the list should be the correct one. Set the baudrate to **115'200** and the configuration as shown below, then click on **Connect**:

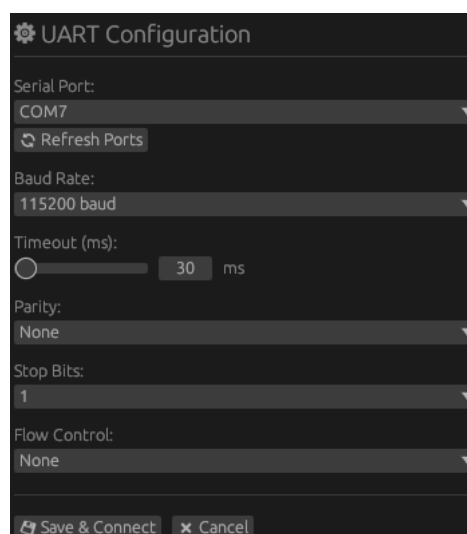


Figure 12 - **UART** configuration



Then ensure the viewer is set to the correct grid size which is **32x32** by default. You can also set input and output folder to read and write results from/to files (optional).

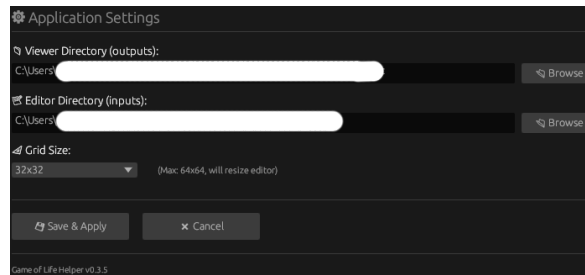


Figure 13 - Grid configuration

3.5.2 Viewing the grid

Switch to the **Viewer** mode and select the **Live UART** option to see grids generated from the ESP board:

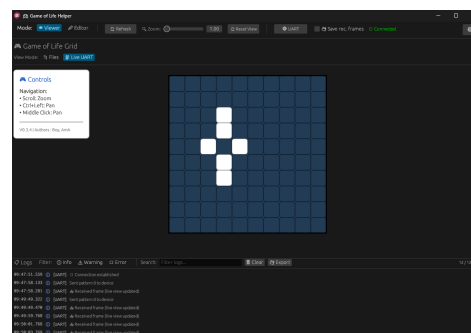


Figure 14 - Live-view

3.5.3 Editing the grid

Switch to the **Editor** to create a custom grid.

Simply left-click on cells to set their state to alive, right-click to set them to dead.

Click the **Classic Mode** to enable the **Color Mode**. Select a color and set cells as usual. Those colors will display on first frame and be carried following your **gol_step** Section 3.2.2 implementation.

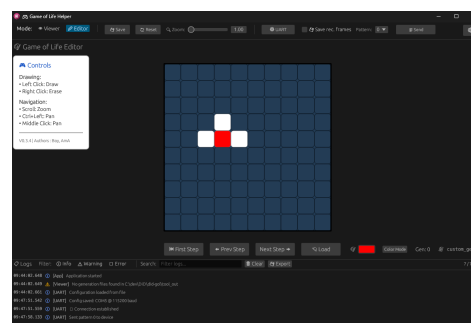


Figure 15 - Editor

Click on the **Send** button in the top right of the viewer to send the grid. Head back to the **Viewer** mode Section 3.5.2 to see the grid evolution.



4 | Evaluation

The final grade includes the report, the code as well as a presentation of the system [Table 1](#).

Evaluated aspects	Points
Report	50
Introduction	2
Specification	2
Project	10
Verification and validation	10
Activity diagram	20
Conclusion	2
Formal aspects of the report	4
Functionality of the circuit	20
Quality of the solution	20
Presentation	20
Total	110

Table 1 - Evaluation grid



The evaluation grid already gives indications about the structure of the report. For a good report, consult the document **How to write a project report?** [\[1\]](#).



Appendix

1 | Debugging (Optional)

1.1 Setup

While not the seamless experience, it is possible to debug the GoL software by stepping through the assembly code while watching registers update. To do so, a VSCode configuration is already set up in the project, using the [Probe-rs Debugger](#) extension.



It is possible to use combination of other editors and viewers using either **probe-rs** or **OpenOCD** to debug the [RISC-V](#) core. However, this is not covered in this documentation.

1.1.1 Extensions

Install the [Probe-rs Debugger](#) extension in VSCode to enable debugging of the [RISC-V](#) core.

1.1.2 Drivers

Drivers must be correctly setup to reach the ESP32 through the USB JTAG/serial debug unit. Even if the ESP32 correctly discuss with the host PC through [UART](#), the debugger uses the **JTAG** port to interact with the core.

For Unix systems, it may work out-of-the-box. If needed, see [Probe-rs setup](#) for more information.

For Windows users, it is required to modify the ESP32 **USB JTAG/serial debug unit** driver through [Zadig](#). The **WinUSB v.6.1** driver is necessary instead of the default one.

Open **Zadig**, select **Options > List All Devices**, then select the **USB JTAG/serial debug unit (Interface 2)**. Click the dropdown near button > **Install device** then click on **Replace Driver**:

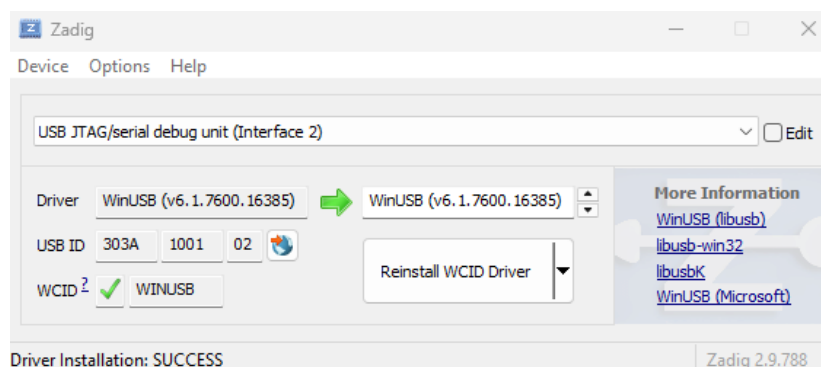


Figure 16 - Zadig driver installation

1.1.3 Debug

Once everything set up, wire the board and:

- Open the viewer with **just gol-viewer**
- Connect the viewer to the correct [UART](#) port - this allows to print messages in the viewer console from the [CPU](#)



Refer to the [Figure 18](#) image:

1. Open the VSCode debugger in the left panel:
2. Select the **Debug GoL** configuration and click on the green play button to build and connect the debugger
3. Set breakpoints in the assembly code in the **GoLRS/src/bin/main.rs** file in front of the line **gol_step** by clicking on the left of the line number (same procedure for **gol_init** if you want to debug the initialization of the grid)
 - Clear logs in the viewer console
4. Click on the blue  button to run the code until the breakpoint is reached
 - Check messages in the viewer and note the given addresses - those correspond to your [ASM](#) code area in the built binary

```
11:04:33.745 [CPU] Initialising grid
11:04:33.745 [CPU] * cur_ptr=0x3fc82e74, nxt_ptr=0x3fc83e74
11:04:33.745 [CPU] * ASM_region=0x4201014c..0x42010494
11:04:33.745 [CPU] * gol_init = 0x42010168
11:04:33.745 [CPU] * gol_step = 0x42010390
11:04:33.745 [CPU] * Initialising grid with blinker oscillator in the centre
11:04:33.745 [CPU] Running GOL main loop
```

Figure 17 - Viewer log

5. Open the control panel with **Ctrl+Shift+P** and run **Open Disassembly View**

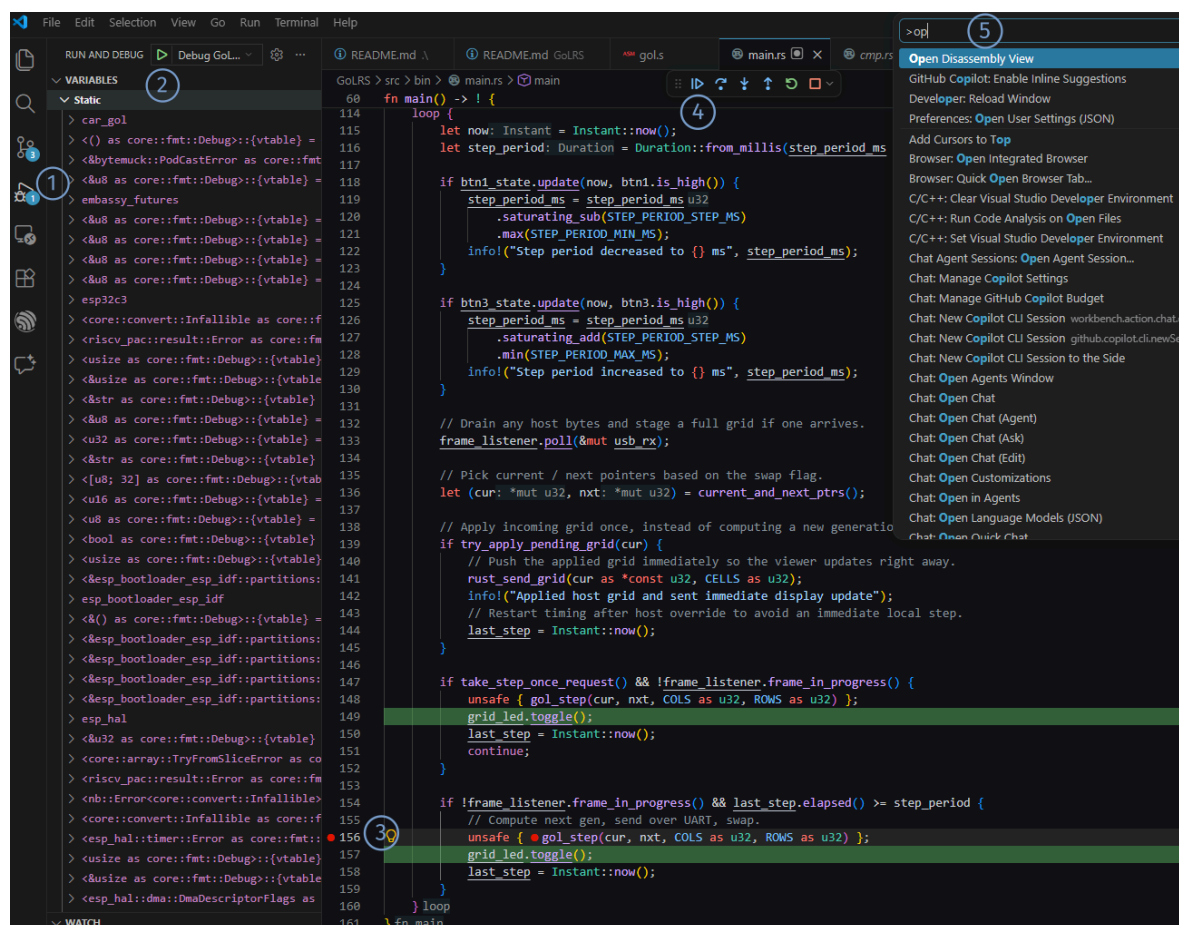


Figure 18 - VSCode Debugger



This opens a new panel with some assembly code. It does not yet correspond to the function you wrote, you still need to iterate a few step until hitting the next **jal** instruction. Either click on the **Step Into** arrow or **F11**:

```

0x420125A0      33 05 65 41      sub a0, a0, s6
539:          self.as_raw() >= 0
0x420125A4      33 35 05 01      sltu a0, a0, a6
0x420125A8      01 4B            c.li s6, 0
0x420125AA      81 4B            c.li s7, 0
0x420125AC      23 2E A4 F5      sw s10, -0xa4(s0)
154:          if !frame_listener.frame_in_progress() && last_step.elapsed() >= step_period {
0x420125B0      E3 12 05 A0      bnez a0, -0x5fc
156:          unsafe { gol_step(cur, nxt, COLS as u32, ROWS as u32) };
0x420125B4      13 06 00 02      addi a2, zero, 0x20
0x420125B8      93 06 00 02      addi a3, zero, 0x20
0x420125BC      66 85            c.mv a0, s9
0x420125BE      A6 85            c.mv a1, s1
0x420125C0      97 E0 FF FF      auipc ra, 0xfffffe
0x420125C4      E7 80 00 DD      jalr ra, ra, -0x230
0x420125C8      37 45 00 60      lui a0, 0x60004
2107:          intrinsics::volatile_load(src)
0x420125CC      48 41            c.lw a0, 4(a0)

```

Figure 19 - Disassembly view

When passing the **jal** instruction, your function should be reached. The left panel shows registers values:

```

VARIABLES
  Static
    <str as core::fmt::Debug>::(vtable)
    <riscv_pac::result::Error as core::fm
    <usize as core::fmt::Debug>::(vtable)
    <usize as core::fmt::Debug>::(vtable)
    <str as core::fmt::Debug>::(vtable)
    riscv_rt
  Registers
    x0/zero = 0x00000000
    x1/LR = 0x420125c8
    x2/SP = 0x3fcae2d0
    x3/GP = 0x3fc8127c
    x4/TP = 0x00000000
    x5/T0 = 0x00ffffff
    x6/T1 = 0x00000000
    x7/T2 = 0x00000003
    x8/FP/S0 = 0x3fc82e74
    x9/S1 = 0x3fc83e74
    x10/A0/R0 = 0x3fc82e74
    x11/A1/R1 = 0x3fc83e74
    x12/A2 = 0x00000020
    x13/A3 = 0x00000020
    x14/A4 = 0x00000000
    x15/A5 = 0x00000000
    x16/A6 = 0x000f4240
    x17/A7 = 0x3fc80d0c
    x18/S2 = 0x40000000
    x19/S3 = 0x60023000
    x20/S4 = 0x60043000
    x21/S5 = 0x3fc81e74
    x22/S6 = 0x00000000
    x23/S7 = 0x00000000
    x24/S8 = 0x00000000
    x25/S9 = 0x3fc82e74
    x26/S10 = 0x000003e8
    x27/S11 = 0x00010000
    x28/T3 = 0x00000001
    x29/T4 = 0x3fc84e70
    x30/T5 = 0x00000000
    x31/T6 = 0x00000000
    PC/PC = 0x420103bc
  WATCH
    0x4201035c
    0x42010360
    0x42010364
    0x42010368
    0x4201036c
    0x42010370
    0x42010374
    0x42010378
    0x4201037c
    0x42010380
    0x42010384
    0x42010388
    0x4201038c
    0x42010390
    0x42010394
    0x42010398
    0x4201039c
    0x420103a0
    0x420103a4
    0x420103a8
    0x420103ac
    0x420103b0
    0x420103b4
    0x420103b8
    0x420103bc
    0x420103c0
    0x420103c4
    0x420103c8
    0x420103cc
    0x420103d0
    0x420103d4
    0x420103d8
    0x420103dc
    0x420103e0
    0x420103e4
    0x420103e8
    0x420103ec
    0x420103f0
    0x420103f4
    0x420103f8
    0x420103fc
    0x42010400
    0x42010404
    0x42010408
    0x4201040c
    0x42010410
    0x42010414
    0x42010418
    0x4201041c
    0x42010420
    0x42010424
    0x42010428
    0x4201042c

```

Figure 20 - Step-by-step execution



Ensure addresses correspond to the range given by the GoLViewer log. Labels do not exist in the disassembly view.

You can then step through the code and watch registers update.



Glossary

ASM – Assembly [9](#), [17](#)

CPU – Central Processing Unit [12](#), [13](#), [16](#)

GoL – Game of Life [3](#), [6](#), [8](#)

HITL – Hardware In The Loop [2](#), [12](#)

ISA – Instruction Set Architecture [5](#)

LED – Light Emitting Diode [3](#)

RGB – Red Green Blue [9](#)

RISCV – 5-stages Reduced Instruction Set Computer architecture, open-sourced [4](#), [11](#), [12](#), [16](#)

UART – Universal Asynchronous Receiver Transmitter [11](#), [12](#), [13](#), [16](#)



Bibliography

- [1] Christophe Bianchi, François Corthay, and Silvan Zahno, “Comment Rédiger Un Rapport de Projet?” 2021.